A CASE-STUDY BASED REPORT

ON

Phishing Website Detection Using Random Forest and NLP-Based URL Analysis

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Monish Putti	**2520030395**
Mokshitha Putti	**2520030396**
Veldi SohithSai	**2520030389**

Under the Esteemed guidance of

Ravindra Gaddam

Asst.Professor, Dept. of CS&IT



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract **3**

Introduction **4**

Background 4
Motivation for the Project 4
Overview of the Work Carried Out 4
Scope and Responsible-Use Statement 5

Problem Statement **6**

Case-Study Scenario 6
Central Case-Study Question 6
Formal Problem Definition 6
Objectives that Guide the Study 6
Assumptions and Limitations 6

Dataset and Exploratory Data Analysis **8**

About the Dataset 8
Purpose of EDA 8
Preparation Steps Actually Performed 9
EDA Exploration - Implementation 9
Summary of EDA Findings 10

Algorithm Selection and Implementation **11**

Conclusion **13**

ABSTRACT

Phishing websites are fraudulent websites designed to steal sensitive user information such as usernames, passwords, banking credentials, and personal data by impersonating legitimate websites. As cyber threats continue to grow, identifying phishing websites quickly and accurately has become an important cybersecurity challenge.

This project proposes a Machine Learning-based phishing website detection system using the Random Forest algorithm combined with Natural Language Processing (NLP)-based URL analysis. The system analyzes various characteristics of URLs such as their length, presence of special characters, suspicious keywords, domain structure, token patterns, and lexical features extracted through NLP techniques. These extracted features are used to train a Random Forest classifier capable of distinguishing legitimate websites from phishing websites.

The proposed model aims to achieve high detection accuracy while maintaining low false-positive rates. The system can assist users by identifying potentially malicious websites before they access them, thereby reducing the risk of online fraud and identity theft.

The project demonstrates how Machine Learning and NLP techniques can be effectively integrated to improve cybersecurity applications and provide a reliable solution for phishing website detection.

INTRODUCTION

Background

Phishing is a form of social-engineering attack in which an attacker creates a website that imitates a trusted brand such as a bank, an e-commerce store or an email provider, and lures victims into entering confidential information. The attack usually begins with an email, SMS or social-media message containing a link. Because the fake page looks very similar to the real one, users often cannot tell the difference at a glance.

Traditional defences such as blacklists work only after a phishing site has been reported, and attackers register new domains every day, so many malicious sites remain undetected during their most active hours. The Uniform Resource Locator (URL) itself carries many clues about its intent: unusually long addresses, many sub-domains, the "@" symbol, hyphens, raw IP addresses and words such as "login", "verify" or "secure" are all common in phishing links. Machine Learning (ML) can learn these patterns automatically from labelled examples and can therefore flag new, previously unseen URLs.

Motivation for the Project

- Phishing is one of the most common causes of credential theft, financial fraud and identity theft, and it affects ordinary users as well as organisations.
- Blacklist-based tools are reactive and cannot keep pace with newly created phishing domains.
- A URL can be analysed instantly without loading the page, which makes URL-based detection fast, lightweight and safe.
- Random Forest is robust, handles mixed numeric features well, resists over-fitting and provides feature-importance scores that make the decision easier to interpret.
- Combining hand-crafted lexical features with NLP-based token analysis lets the model capture both the structure and the "language" of a malicious URL.

Overview of the Work Carried Out

The work follows the standard machine-learning workflow for a supervised classification task:

1. Collect a labelled dataset of phishing and legitimate URLs.
2. Clean and pre-process the URLs (removal of duplicates, normalisation, handling of missing values).

3. Extract lexical, domain-based and NLP-based (token and keyword) features from every URL.
4. Explore the data through exploratory data analysis (EDA) to understand class balance, feature distributions and correlations.
5. Split the data into training and testing sets and train a Random Forest classifier, with Logistic Regression and Decision Tree as baselines.
6. Tune hyper-parameters and evaluate the model using accuracy, precision, recall, F1-score, ROC-AUC and the false-positive rate.
7. Analyse feature importance and draw conclusions about the most reliable phishing indicators.

Scope and Responsible-Use Statement

The scope of this study is limited to URL-based detection of phishing websites using supervised machine learning. Page content, screenshots, WHOIS records and live network traffic are outside the scope of the current work. The system is built purely for educational and defensive purposes: to help users and security teams identify malicious links. Only publicly available, non-personal URL data is used, no real user credentials are collected, and no phishing pages are created, hosted or tested against real victims. The model produces a prediction, not a guarantee, and should be used as one layer of a wider security strategy together with user awareness and other safeguards.

PROBLEM STATEMENT

Case-Study Scenario

An online-banking and e-commerce user receives an email that says, "Your account has been suspended. Verify your details immediately," along with a link such as http://secure-login.account-verify.example-bank.com.xyz/update?id=8372. The page behind the link looks identical to the real bank website. If the user enters credentials, they are sent to the attacker. A security team wants an automatic system that can examine the link before the user opens it and warn the user if the link is likely to be phishing, while avoiding false alarms on genuine websites.

Central Case-Study Question

Can a Random Forest classifier, trained on features extracted from the URL using lexical analysis and NLP techniques, accurately distinguish phishing websites from legitimate websites while keeping the false-positive rate low?

Formal Problem Definition

Let U = {u1, u2, ..., un} be a set of URLs, where each URL ui is described by a feature vector xi = (x1, x2, ..., xm) obtained through lexical, domain-structure and NLP-based token analysis, and let yi belong to {0, 1} be its label (0 = legitimate, 1 = phishing). The task is to learn a function f : X -> {0, 1}, implemented as a Random Forest ensemble of decision trees, that predicts the label of an unseen URL so that the classification error, and in particular the number of legitimate sites wrongly flagged as phishing, is minimised.

Objectives that Guide the Study

1. To extract meaningful lexical, domain-structure and NLP-based features from URLs.
2. To build a Random Forest classifier that separates phishing URLs from legitimate URLs.
3. To achieve high detection accuracy while maintaining a low false-positive rate.
4. To compare the Random Forest model with baseline classifiers such as Logistic Regression and Decision Tree.
5. To identify the most influential features using feature-importance analysis.
6. To demonstrate how ML and NLP can be integrated into a practical cybersecurity solution that warns users before they visit a malicious website.

Assumptions and Limitations

Assumptions

- The labelled dataset is reasonably representative of real-world phishing and legitimate URLs.
- The labels (phishing / legitimate) in the dataset are correct.
- A URL carries enough information, on its own, to indicate whether a site is malicious.
- Features are computed from the URL string only, without visiting the website.

Limitations

- URL-only analysis can miss phishing sites that use clean-looking domains, compromised legitimate sites or newly registered look-alike domains.
- Attackers may adapt their URLs to avoid known patterns (concept drift), so the model needs periodic retraining.
- Shortened URLs hide the real destination unless they are expanded first.
- Performance depends on the quality and balance of the dataset, and results on one dataset may not carry over exactly to another.
- The system does not analyse page content, images or SSL-certificate details.

DATASET AND EXPLORATORY DATA ANALYSIS

About the Dataset

The study uses a labelled collection of URLs in which each record is marked as either phishing (1) or legitimate (0). Such datasets are compiled from public sources: phishing URLs from verified feeds such as PhishTank or OpenPhish, and legitimate URLs from popular-domain rankings such as Tranco or the Alexa/Majestic lists; ready-made versions are also available on Kaggle and the UCI repository. The raw data contains the URL string and the class label, and the features described below are derived from them.

Item	Description
Type of data	Labelled URLs (text) with a binary class label
Target variable	label: 1 = phishing, 0 = legitimate
Input	Raw URL string, converted into numeric feature vectors
Task	Supervised binary classification
Number of records / features	[Insert after loading the chosen dataset]

Table 1: Dataset overview

Features are grouped into three families, as listed below.

Feature group	Examples
Lexical features	URL length, hostname length, path length, number of dots, hyphens, digits, "@", "?", "=", "&", "%" and "/" characters, digit-to-letter ratio
Domain-structure features	Presence of an IP address instead of a domain name, number of sub-domains, top-level domain (TLD), use of HTTPS, use of URL shorteners, depth of the path
NLP-based features	Tokens obtained by splitting the URL on delimiters, count of suspicious keywords (login, verify, secure, account, update, bank, signin, confirm), brand names appearing outside the real domain, character n-gram / TF-IDF token weights, URL entropy

Table 2: Feature groups extracted from each URL

Purpose of EDA

Exploratory Data Analysis is carried out before modelling to understand the structure and quality of the data. In this project EDA is used to check the balance between phishing and legitimate classes, find missing values and duplicate URLs, study how each feature differs between the two classes, detect outliers such as extremely long URLs, and examine correlations between features

so that redundant features can be identified. These insights guide feature selection and the choice of evaluation metrics.

Preparation Steps Actually Performed

1. Loaded the dataset with pandas and inspected its shape, data types and first records.
2. Removed duplicate URLs and rows with missing URL or label values.
3. Normalised URLs (lower-casing, trimming spaces, removing the protocol prefix where required).
4. Parsed every URL into scheme, hostname, sub-domain, domain, TLD, path and query components.
5. Extracted lexical and domain-structure features and computed NLP features by tokenising the URL on special characters and counting suspicious keywords.
6. Converted tokens into numeric form using TF-IDF / character n-grams and encoded categorical items such as the TLD.
7. Split the data into 80% training and 20% testing sets using a stratified split to preserve the class ratio.

EDA Exploration - Implementation

The EDA is implemented in Python using pandas, NumPy, Matplotlib and Seaborn. The main analyses are:

- **Class distribution:** a count plot of phishing versus legitimate URLs to check for class imbalance.
- **Univariate analysis:** histograms and box plots of URL length, number of dots, digits and special characters.
- **Bivariate analysis:** comparison of each feature between the two classes, for example average URL length of phishing versus legitimate sites.
- **Correlation analysis:** a heat-map of feature correlations to detect highly correlated features.
- **Keyword analysis:** frequency of suspicious keywords in each class.

```
import pandas as pd, seaborn as sns, matplotlib.pyplot as plt
df = pd.read_csv("urls.csv")
print(df.shape, df.isnull().sum(), df.duplicated().sum())
sns.countplot(x="label", data=df)            # class balance
sns.boxplot(x="label", y="url_length", data=df)
sns.heatmap(df.corr(numeric_only=True), cmap="coolwarm")
```

Summary of EDA Findings

The EDA is expected to confirm the following patterns, which are well documented for phishing URLs and which should be verified against the chosen dataset:

- Phishing URLs tend to be longer and to contain more special characters (hyphens, "@", "?", "=") than legitimate URLs.
- Phishing URLs more often contain raw IP addresses, many sub-domains and suspicious keywords such as "login", "verify" and "secure".
- Legitimate URLs are usually shorter and use well-known domains with simple structure.
- Some features are strongly correlated (for example URL length and path length), so redundant features may be dropped.
- If the classes are imbalanced, class weights or resampling are used and accuracy is reported together with precision, recall and F1-score.

ALGORITHM SELECTION AND IMPLEMENTATION

Why Random Forest

Random Forest is an ensemble learning method that builds many decision trees on random subsets of the training data (bootstrap sampling) and random subsets of the features, and then combines their predictions by majority voting. It was selected for this project for the following reasons:

- It handles numerical, count and encoded categorical features together without heavy scaling.
- Averaging many trees reduces variance and the risk of over-fitting compared with a single decision tree.
- It performs well on high-dimensional, non-linear data such as URL features.
- It is robust to noisy features and outliers.
- It provides feature-importance scores, which help explain which URL characteristics indicate phishing.

Working of the Algorithm

1. Draw N bootstrap samples from the training data.
2. For each sample, grow a decision tree; at every split consider only a random subset of features and choose the best split using Gini impurity (or entropy).
3. Grow each tree to its maximum allowed depth.
4. To classify a new URL, pass its feature vector through all trees.
5. Output the class (phishing or legitimate) that receives the majority of votes.

Implementation

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

rf = RandomForestClassifier(class_weight="balanced", random_state=42, n_jobs=-1)
grid = GridSearchCV(rf, {"n_estimators": [100, 200, 300],
                         "max_depth": [None, 10, 20],
                         "min_samples_split": [2, 5]},
                    cv=5, scoring="f1")
grid.fit(X_train, y_train)

best = grid.best_estimator_
pred = best.predict(X_test)
print(classification_report(y_test, pred))
print(confusion_matrix(y_test, pred))
print("ROC-AUC:", roc_auc_score(y_test, best.predict_proba(X_test)[:, 1]))
importances = best.feature_importances_
```

Hyper-parameters Tuned

Parameter	Meaning	Values tried
n_estimators	Number of trees in the forest	100, 200, 300
max_depth	Maximum depth of each tree	None, 10, 20
min_samples_split	Minimum samples needed to split a node	2, 5
class_weight	Handles class imbalance	balanced

Table 3: Hyper-parameter search space

Evaluation Plan

The tuned model is evaluated on the unseen test set using accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix. Because wrongly blocking a genuine website is costly, the false-positive rate (legitimate sites predicted as phishing) is reported explicitly. Logistic Regression and Decision Tree classifiers are trained on the same features as baselines, so the benefit of the Random Forest can be shown fairly. Finally, the feature-importance ranking is plotted to show which URL characteristics contribute most to detection.

CONCLUSION

This case study presented a phishing website detection system that combines NLP-based URL analysis with a Random Forest classifier. By converting each URL into lexical, domain-structure and token-based features, the model learns to separate phishing links from legitimate ones without needing to open the website, which makes detection fast and safe for the user. Random Forest suits the problem because it is accurate, resistant to over-fitting and interpretable through feature importance.

The approach aims at high detection accuracy with a low false-positive rate, helping to reduce online fraud and identity theft. The main limitations are the dependence on dataset quality and the fact that attackers continually change their tactics, so the model should be retrained regularly. Future work can add page-content and WHOIS/domain-age features, deep-learning text models such as character-level CNNs or transformers, and deployment as a browser extension or web service that checks links in real time.

Signature of the Team members